Journal of Information Processing Vol.31 174–184 (Mar. 2023)

[DOI: 10.2197/ipsjjip.31.174]

Recommended Paper

CREBAS: Enabling Network Access Control in a Home with One Click

Naoki Matsumoto[1,a)] Daisuke Kotani[1,b)] Yasuo Okabe[1,c)]

Received: April 8, 2022, Accepted: December 2, 2022

Abstract: Many smart home devices, such as smart speakers, are connected to a home network. Conventional home networks do not control communications among devices, including malicious devices. As a result, malicious devices in a home network can steal private information and manipulate devices that have no or weak access control mechanisms. Some methods have been proposed to protect devices, but cannot control communications in some situations and do not discuss their authorization architecture or usability. Malicious devices can bypass access control, and users may make incorrect configurations due to their complexity. They can make access control meaningless. In this paper, we propose CREBAS, which enables network access control to protect devices from malicious or unintended communications. We propose a capability-based authorization architecture and discuss its validity according to the guidelines for access control in a home. Also, we propose methods that provide more fine-grained and stricter network access control according to authorized capabilities. We have implemented a prototype of CREBAS. The prototype has shown that the performance degradation is acceptable, and it is possible to perform access control with one click in a real protocol used in a home network.

Keywords: network access control, home network, capability model, smart home device

1. Introduction

Home networks have many devices, which are not only conventional devices such as PC but also smart home devices such as smart speakers and smart TVs. These smart home devices work together through a home network and allow us to control and check the status of home appliances with terminals in our hands or voice. Currently, most devices work together with external systems in a cloud, but more devices will cooperate by direct communications [6] in a home network.

Conventional home networks do not have methods to control communication between devices or have only simple limited security mechanisms. Devices can communicate with each other without any limitations. Therefore, all devices in a home network are exposed to the risk of attacks or unintended communication by malicious devices. Malicious devices can search a home network and intercept private information. In addition, these devices can operate other devices and allow them to behave in unintended ways for users. As a result, a lack of security mechanisms in a home network can cause not only the leaking of private information but also serious matters which affect users' lives.

In the case of cooperation with direct communications, not all devices support access control, and access control mechanisms can be limited due to the limitations of software or computing resources of devices. Actually, some communication protocols [7], [8] are not designed to perform authentication or access control. As a result, attackers and malicious devices can easily exploit devices and steal information from them. As more devices and communication protocols are used in a current home network, these security problems will become more serious.

To protect devices from attack, network access control methods [4], [5] have been proposed. These methods control a network using OpenFlow [9]. Although these methods claim that they can protect devices from malicious or unintended attacks, they rely on flow-based access control, so they cannot provide access control for messages transferred via TCP streams. This means that these methods can remove completely malicious devices, but cannot prevent devices that work correctly from scanning the network and stealing information covertly.

They showed that network access control technically works as mitigation for vulnerabilities, but they did not discuss their authorization architecture. An authorization architecture is a system used by users to configure access control. Without a properly designed authorization architecture, it may be difficult for users to make appropriate configurations for each device and check whether the access control is applied as intended. Users may make incorrect configurations, which allow malicious communications. This means that no matter how excellent a network access control method is, it is meaningless if the authorization architecture is not properly designed. An appropriate authorization architecture is essential for an access control method, especially in a home.

[1] Kyoto University, Kyoto 606–8501, Japan
[a)] mt2.naoki@net.ist.i.kyoto-u.ac.jp
[b)] kotani@media.kyoto-u.ac.jp
[c)] okabe@i.kyoto-u.ac.jp

The preliminary version of this paper was published at Multimedia, Distributed, Cooperative, and Mobile Symposium (DICOMO 2021) in July 2021. The paper was recommended to be submitted to Journal of Information Processing (JIP) by the chief examiner of SIGIOT.

© 2023 Information Processing Society of Japan

Some research on usable access control and an authorization architecture in a home [10], [11], [12], [13], [14], [15], [16] are performed. These works consider the requirements for more usable authorization architectures in various home environments. To bring these works into network access control, we need to address some problems. Home networks have not only devices that support new access control methods but also existing devices that do not support them. Network access control methods can provide access control for both devices without any modifications, but authorization architectures require devices to support their protocols. Also, there is a gap in the granularity of access control between network access control methods and authorization architectures. Authorization architectures provide access control for each operation or content, but as we mentioned, network access control methods using flow-based control cannot provide such fine-grained control. We solve these problems by considering both an authorization architecture and network access control methods.

In this paper, we propose CREBAS, which enables network access control in a home with one click. **Figure 1** shows the overview of CREBAS. CREBAS aims to control each communication in units of operation like `Get Temperature` or `Turn on Light`. Providing fine-grained access control increases complexity and burden on users. To reduce such a burden, CREBAS has an authorization architecture based on the capability model, which enables users to achieve fine-grained authorization easily, and enables network access control methods to achieve control communications in units of operation in a home with the authorized capability.

First, we propose an authorization architecture based on the capability model. We designed the authorization architecture according to the guidelines [11]. The capability model authorizes access to each device's communication or function and issues the token called capability. Capability tokens prove rights to use or access the specified resources. This model achieves fine-grained authorization and is suitable for home networks where various devices exist. This is because capabilities are assigned to each device's communication and users authorize it to control communications. Users are not required to investigate device specifications and write policies such as ACL. But it requires frequent operations to authorize capabilities and it is a burden on users. To reduce the number of requests to request users to make authorization decisions, our authorization architecture incorporates Automatic Authorization. Automatic Authorization makes authorization decisions automatically on behalf of the user.

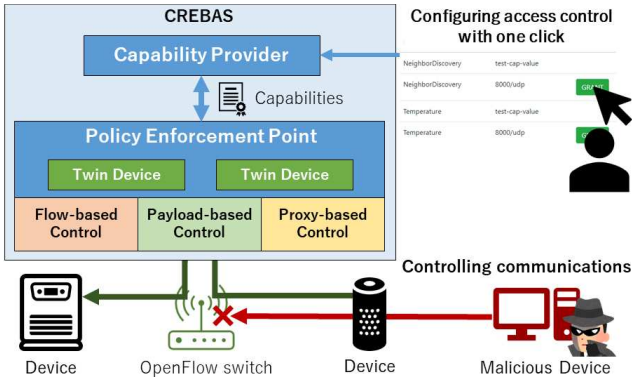


Fig. 1 The overview of CREBAS.

Secondly, we propose network access control methods that control communications according to the authorized capabilities. This method enables more fine-grained access control through flow-based control, payload-based control, and proxy-based control. Existing devices cannot handle capabilities and cannot inspect the packet's payload. To solve this problem, we designed a twin device that complements such missing functions. The twin device enables our proposals to integrate into a home network without major modifications to existing devices.

Then, we implemented a prototype of CREBAS. We confirmed that the performance degradation is acceptable and it is possible to control communications finely among smart home devices.

Finally, we discuss the validity in terms of the usability of our authorization architecture.

The following is the structure of this paper. Section 2 describes the related works, Section 3 describes the capability-based authorization architecture, Section 4 describes the network access control method, Section 5 describes the implementation of the prototype system and the evaluation, Section 6 discusses the authorization architecture, and finally, Section 7 summarizes this paper.

2. Related Works

2.1 Network Access Control in a Home

Table 1 shows the comparison with other network access control methods.

Feamster [1] showed that it is possible to detect spam in e-mail and botnets by outsourcing the management of home networks using OpenFlow. OpenFlow can provide flow-based access control from a remote controller. Flores et al. [2] designed a platform to control home networks with NFV-based applications. Osman et al. [3] proposed an automatic control system. The system analyzes devices in a home network with an analysis system built on a cloud and separates the network automatically using OpenFlow. These works propose to delegate the management of the network from users who lack knowledge of networks to external

Table 1 Comparison with other network access control methods.

	(1) Configuring by users	(2) Independent system	(3) Flow-based control	(4) Message-based control	(5) Discussion of usability
Feamster [1]			✓		
Moyano et al. [2]			✓		
Osman et al. [3]			✓		
Mortier et al. [4]	✓	✓	✓		
Lear et al. [5]	✓		✓		
CREBAS	✓	✓	✓	✓	✓

© 2023 Information Processing Society of Japan

experts or automated mechanisms. In these works, they argue that a fully automated system is easy to use because it eliminates user hassles. However, according to Yao et al.'s work [10], users are reluctant to delegate the management of devices in the home to an external party. A system in which users do not have control is not desirable. A mechanism or interface for (1) Configuring by users is necessary, but it is not discussed. Also, these methods have to send information about each individual's device and network to the external system, which causes privacy problems. In addition, if the external system that manages the network becomes unavailable, the home network also becomes unavailable.

On the other hand, there are some (2) Independent systems without outsourcing. These systems have no privacy and availability issues. Mortier et al. [4] proposed a system to manage devices in a home network by integrating routers. Manufacturer Usage Description (MUD) [5] enables vendors to define specifications of communications and access control policies for each IoT device and apply them automatically. MUD controls communications among devices so that MUD can protect devices even if devices have vulnerabilities.

Outsourcing and standalone methods both use only (3) Flow-based control. Flow-based control has a limitation in terms of the granularity of control. If a device performs both authorized and malicious communications in the same TCP stream, flow-based control methods cannot control it properly.

Although these methods showed technical possibilities to control home networks, they did not discuss or experiment with their usability. The discussion about usability is important because the methods are applied to real home networks and users. If users make incorrect configurations, access control will not work. There are some works on access control for ordinary users in a home [10], [11], [12], [13], [14], [15], [16]. (5) Discussion of usability should be performed to design a more usable system and evaluate its system validity.

In our research, we propose CREBAS. CREBAS is an independent system that does not delegate control to external parties and consists of authorization architecture and network access control methods. The authorization architecture is designed based on the guidelines [11] for access control in a home network, and we discuss the validity in terms of its usability. Users can configure the access control with one click on the CREBAS WebUI. To provide more fine-grained network access control, we propose (4) Message-based control methods called *payload-based* and *proxy-based* access control. These methods enable to allow authorized messages and block malicious messages in the same TCP stream, which cannot be achieved with flow-based control.

2.2 Capability-based Access Control

The capability model [17] is a kind of access control model. In this model, a device issues a capability to a user, and the user can use the corresponding function by presenting the capability to the device. Gusmeroli et al. [18] proposed an access control for IoT devices based on the capability model. In the case of Role-Based Access Control and Attribute-Based Access Control, there were problems such as role explosion and difficulty in ensuring consistency of attributes. Capability-Based Access Control (CapBAC) solves these problems and manages access for each device easily. Additionally, by delegating capabilities to others, they can authorize and issue delegated capabilities. However, these CapBAC methods cannot be applied to existing devices because they require devices to process capability and control access. Various devices are produced by various vendors and need to support the same CapBAC method. CREBAS provides CapBAC without modifications to devices including existing devices by controlling communications on the network side.

3. Authorization Architecture

3.1 The Guidelines for Access Control at Home

Mazurek et al. worked out the guidelines [11] shown below for access control systems aimed at home users.

(1) Allow fine-grained control
(2) Plan for lending devices
(3) Include reactive policy creation [12]
(4) Include logs
(5) Reduce or eliminate up-front complexity
(6) Support iterative policy specification

We propose the authorization architecture based on the capability model to meet these guidelines. Conventional policy-based access control makes it difficult for users to achieve these guidelines. More detail is discussed in Section 6.

3.2 The Overview of Authorization Architecture

The authorization architecture is designed based on the capability model. In this architecture, each communication is represented as a capability. A capability represents the purpose and use of communication in a way that users can easily understand. For a device to communicate with other devices, an appropriate capability must be issued to the device. This architecture has two authorization flows, *Automatic Authorization* and *Manual Authorization*. Following either authorization flow, capabilities are authorized and issued to devices.

Figure 2 shows the overview of the authorization architecture. Each device has capabilities shown in **Fig. 3**. capabilityName represents the purpose and the use of communication. Users check this field and decide whether to authorize the capability or not. capabilityGrantPolicy is an encoded authorization policy used in Automatic Authorization. capabilitySignature is a signature signed by a device that provides the capability when it delegates the capability. Capabilities are also signed when Capability Provider (CP) issues them to devices or delegates them to

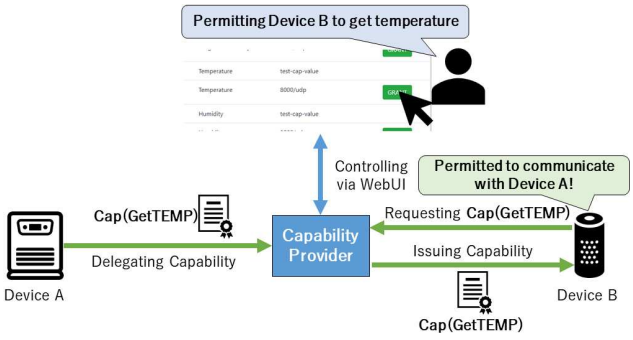


Fig. 2 The overview of authorization architecture.

© 2023 Information Processing Society of Japan

```
{
  "capabilityID":"004bb012-3e39...",
  "assignerID":"ef4bfdac-5760...",
  "assigneeID":"23c186a6-8ec9...",
  "capabilityName":"GetTemperature",
  "capabilityValue":"80/tcp",
  "authorizeCapabilityID":"e1ee77ff...",
  // encoded authorization policy
  "capabilityGrantPolicy":"ehoIeLw...",
  "capabilitySignature":{
    "signerID":"ef4bfdac-5760...",
    "signeeID":"23c186a6-8ec9...",
    "signature":"gI+wYKnYxN/XI0wWh..."
  },
  ...
}
```

Fig. 3 An example of capability.

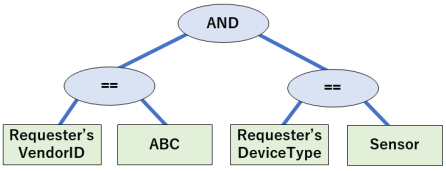


Fig. 4 An example of binary expression tree.

users. Devices and CP check the signature when they apply the capability.

Capability Provider (CP) manages and authorizes the capabilities of devices. All capabilities of the devices are delegated to CP. CP issues capabilities to devices that request them when they are authorized with Automatic Authorization. When they are authorized with Manual Authorization, CP delegates capabilities to users. Users can check the status of the authorizations and authorize capabilities manually using the WebUI implemented in CP.

Each device must have a function to process capabilities. Most devices do not have such a function. Also, it is hard to implement the function in existing devices. To solve this problem, we use *twin device* which is described in Section 4.

3.3 Automatic Authorization

In the capability model, the number of requests to authorize a capability can increase because all communications must be authorized by users. To reduce the number of requests to authorize, CP has a mechanism called *Automatic Authorization* using capability delegation. In Automatic Authorization, capabilities are authorized and issued automatically without the user's operations. Capabilities have a built-in authorization policy to use Automatic Authorization. An authorization policy decides whether to authorize based on the requester's information. When a device requests a capability, CP checks an authorization policy in a capability and authorizes it automatically or delegates the capability to a user for Manual Authorization.

An automatic authorization policy uses a Binary Expression Tree to represent a conditional expression. An example tree shown in **Fig. 4** represents condition (Requester's VendorID == ABC) AND (Requester's DeviceType == Sensor). CP decodes the authorization policy and checks the requester's attributes to determine whether the requester meets the condition or not. If the requester meets the condition, CP authorizes and issues the capability to the requester.

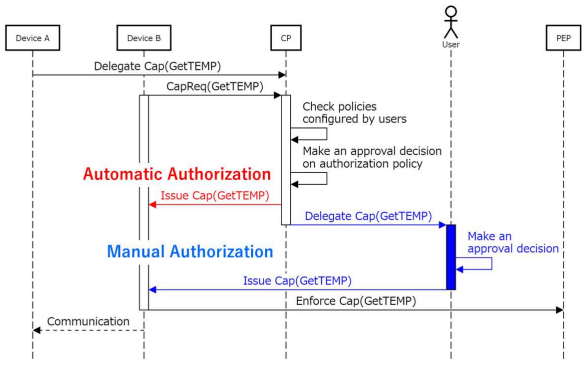


Fig. 5 Flows of Authorization.

Some malicious devices may exploit Automatic Authorization, and capabilities can be issued unintentionally. To prevent exploitation, users can override Automatic Authorization by configuring policies not to authorize capabilities automatically. The policy can specify which capabilities and what requests from devices are not automatically authorized.

3.4 Flows of Authorization

Automatic Authorization authorizes and issues capabilities without the user's intervention. Manual Authorization treats capability requests which are not handled by Automatic Authorization. Users must authorize and issue capabilities themselves.

As an example, we define the capability required to obtain the value of temperature from the device as Cap(GetTEMP). Also, we define the capability request to request Cap(GetTEMP) as CapReq(GetTEMP).

A) Automatic Authorization

The flow of Automatic Authorization follows as shown in **Fig. 5**. Device A delegates Cap(GetTEMP) to the CP and Device B requests it with CapReq(GetTEMP). Cap(GetTEMP) has an authorization policy that allows it to be authorized when requested by Device B. CP automatically issues Cap(GetTEMP) to Device B.

B) Manual Authorization

The flow of Manual Authorization follows as shown in Fig. 5. Cap(GetTEMP)'s authorization policy does not permit it to be authorized automatically. Device B requests it with CapReq(GetTEMP), but Automatic Authorization fails. CP delegates Cap(GetTEMP) to a user and asks whether the user authorizes it. If the user authorizes, the user issues a Cap(GetTEMP) to Device B. In Manual Authorization, capabilities are once delegated to CP and redelegated to users. Capabilities are issued by users, so it is easy to check who authorized the capabilities.

4. Network Access Control

4.1 The Overview of Network Access Control

The objective of network access control is to control communications between devices according to authorized capabilities. Previous works [1], [2], [3], [4], [5] used flow-based control with OpenFlow. However, the granularity of flow-based control is limited to *flow*. This limitation allows malicious devices to bypass access control. For example, when a device is allowed to send messages via the same TCP stream, the device can send both authorized and malicious messages. This is because flow-based

© 2023 Information Processing Society of Japan

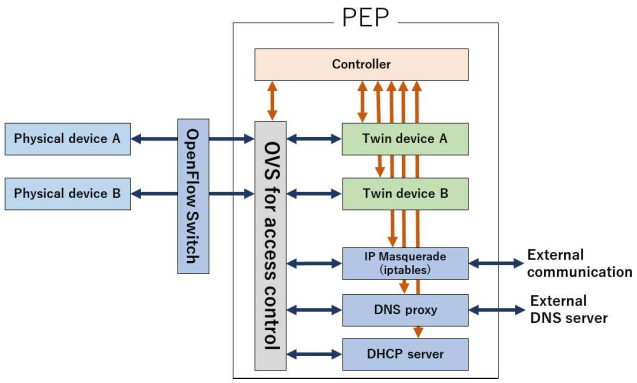


Fig. 6 The overview of PEP.

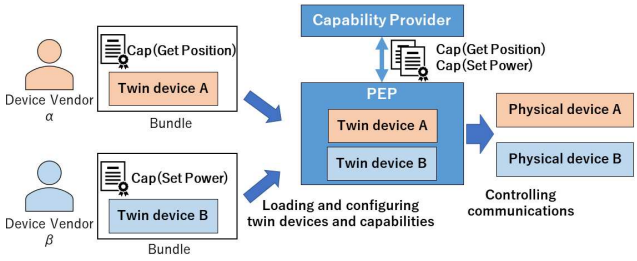


Fig. 7 Loading twin devices.

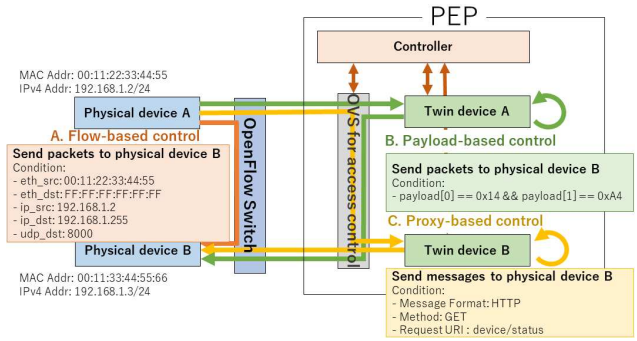


Fig. 8 Access control methods.

control relies on 5-tuples in a packet so that it cannot distinguish between authorized and malicious messages transferred via TCP streams. We propose three access control methods, flow-based, payload-based, and proxy-based access control. These methods enable more fine-grained and sufficient granularity in access control. They can even control messages transmitted through the same TCP stream.

We designed the Policy Enforcement Point (PEP) to control communications between devices with three access control methods. **Figure 6** shows the overview of PEP. A Controller controls components in PEP and an external OpenFlow switch. OVS for access control is Open vSwitch (OVS) in PEP. It forwards packets to an indented twin device or an OpenFlow switch. Twin device is software to process capabilities and provides proxy-based and payload-based control on behalf of a physical device. DHCP server is used to manage IP addresses for physical devices and configure twin devices. IP Masquerade and DNS proxy are used to control external communication. CREBAS provides domain-based access control for external communications. Capabilities used for control of external communications have a field like `*.aaa.example.com` to specify domains. If a capability is authorized, the controller configures IP Masquerade to allow communications to the domain's IP addresses. The endpoint IP addresses can vary, and we need to know the relationship between domains and IP addresses. DNS proxy handles DNS queries and notifies the relationship to the controller.

Some malicious devices may attack these components to disable access control. To prevent such attacks, OVS for access control controls all flows of the components. DHCP server, DNS proxy, and devices connected to the PEP host cannot access the host, and only the minimum required communications among them are allowed.

4.2 A Twin Device

A twin device processes capabilities and provides access control on behalf of physical devices. Existing devices cannot handle capabilities and do not implement access control methods. A twin device fills the missing functions in these devices.

The substance of a twin device is a binary and a process. The controller sends authorized capabilities for a physical device to the corresponding twin device. A twin device recognizes received capabilities and checks whether UDP packets and TCP messages from other devices are allowed or not. The code for a twin device is written in ordinary programming language and there are no limitations in expressions. Therefore, twin device developers can define rules and filters freely as they want.

Each twin device is separated from the PEP host and other twin devices by the Linux network namespace (netns). A twin device has a veth connecting between its netns and OVS for access control. This veth device is used to process packets controlled with payload-based or proxy-based methods. To forward packets to twin devices, capabilities specify an access control method, protocol, and a TCP or UDP port. If a capability is defined to use flow-based control, the controller configures OpenFlow switches to allow corresponding flows. If a capability is defined to use payload-based control, the controller configures switches to forward the corresponding packets to the twin device. If a capability is defined to use proxy-based control, the controller configures switches to forward and rewrite headers for TCP transparent proxy in a twin device. Twin devices can receive and send only packets related to their devices.

Device vendors can control their original protocol by developing their own twin devices. As shown in **Fig. 7**, PEP can provide access control for vendors' original protocols by simply loading and configuring twin devices.

PEP needs to configure twin devices to control access. Using a dedicated DHCP server, PEP automatically configures a twin device. Users are not required to configure the twin devices themselves. A physical device sends a DHCP request with ClientIdentifier. DHCP server orders PEP to configure a twin device. PEP identifies and configures a twin device based on ClientIdentifier.

4.3 Access Control Methods

Figure 8 shows our proposed flow-based, payload-based, and proxy-based control methods.

A) Flow-based Control Method

Flow-based control uses OpenFlow to control flows. In our

© 2023 Information Processing Society of Japan

proposal, devices are connected to an OpenFlow switch or an OpenFlow wireless access point. If an appropriate capability is presented, the controller in a PEP inserts flows into an OpenFlow switch to allow corresponding communications. The devices can perform communications related to the capability, but the other communications are denied.

However, as mentioned above, this method cannot protect the device if a malicious device performs both authorized and unauthorized malicious communications to the same port on the target device. The following two methods solve this problem.

B) Payload-based Control Method

Payload-based control inspects packets' payload. Some protocols, such as UPnP and SSDP, use a single UDP packet to discover devices in a network. For example, a situation can happen where some services are permitted to discover devices, but other services are not permitted. Access control is based on authorized capabilities, and we need to provide a method to control these discoveries. In payload-based control, an OpenFlow switch directs packets to twin devices. The twin devices inspect the payload of the packet and pass or discard packets. In the implementation, a twin device performs recv(2) and send(2) with a raw socket. A twin device parses packets and passes or discards according to the authorized capabilities. Some packets are broadcasted, and it is difficult to control them in a network using an OpenFlow switch. Therefore, payload-based control inspects and controls the *response* packet. If the response packet is discarded, the device is not discovered.

C) Proxy-based Control Method

Proxy-based control inspects messages transferred by TCP. HTTP, which uses TCP, is often used for communication between devices. For example, some protocols [8], [19] are defined using the HTTP REST API. In the REST API, each operation is represented with an HTTP method and a URL, and capabilities are assigned to each operation. Messages including HTTP requests and responses need to be controlled according to authorized capabilities. Proxy-based control parses, checks, and passes or discards messages using TCP proxy in twin devices.

An OpenFlow switch rewrites a packet's header to a temporary address to direct packets to the TCP proxy. The TCP proxy executes recv(2) to retrieve a message, checks it, and executes send(2) to pass it or discard it.

4.4 The Flow of Access Control

Figure 9 shows the flow of access control. Physical devices and twin devices are configured with DHCP. Twin devices present a capability authorized by CP or a user to the controller in PEP. The controller adds flows generated from the capability to an OpenFlow switch. The controller then orders the twin devices to perform payload-based or proxy-based control according to the presented capability. Finally, authorized communications become available, and the other communications are blocked.

4.5 A Trick Forwarding Packets

In payload-based and proxy-based control, packets are forwarded to PEP. In Fig. 8, Physical device A (Device A) sends packets. These packets must be forwarded to a twin device in

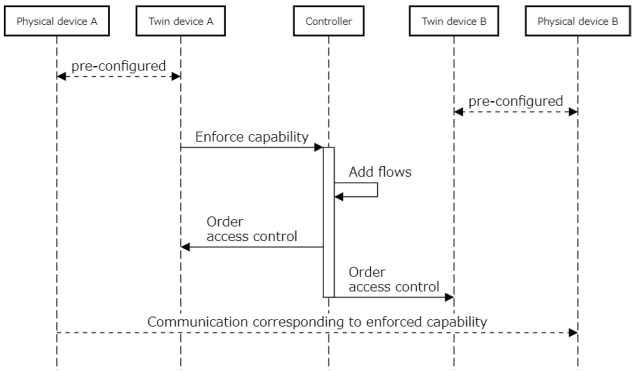


Fig. 9 The flow of network access control.

PEP. However, if an L2 switch exists between an OpenFlow switch and a PEP host, an L2 switch cannot forward packets to the PEP host correctly. Packets sent from Device A have Device A's ethernet address in the field of the source ethernet address. An L2 switch learns that Device A is connected to the port to which the OpenFlow switch is connected. Then, Physical device B (Device B) sends packets to Device A with Device A's ethernet address in the field of the destination ethernet address. The OpenFlow switch tries to forward packets to the PEP host. According to the learned ethernet address table, an L2 switch forwards these packets to the port to which the OpenFlow switch is connected. As a result, packets from Device B to Device A are forwarded to Device A. Packets are not forwarded to the PEP host, and payload-based and proxy-based control do not work.

We solved this problem by rewriting a packet's ethernet address. For packets from Device A to PEP, the OpenFlow switch rewrites the Ethernet Destination field from the Device B's address to the PEP host's address. PEP knows the relationship between an IP address and an ethernet address of a device. OVS for access control rewrites Ethernet Destination field from a PEP host's address to a Device B's address based on a destination IP address. Then, packets are processed in PEP. For packets from PEP to Device B, the OVS for access control rewrites Ethernet Source field from the Device A's address to the PEP host's address. The OpenFlow switch rewrites the Ethernet Source field from a PEP host's address to a Device A's address based on a source IP address. In this way, an L2 switch learns the ethernet addresses correctly and packets are forwarded to the PEP host.

5. Implementation and Evaluation

We implemented a prototype of CREBAS. Using the system, we measured the performance in a real environment and confirmed that the methods have little impact on performance in a home network. Furthermore, we confirm that CREBAS can provide more fine-grained access control than flow-based control methods in a real protocol.

5.1 Prototype Implementation

Figure 10 shows the WebUI of the prototype implementation. All capability requests, pending authorization capabilities, and granted capabilities are listed. Users can authorize capability manually by simply checking capabilityName and clicking the GRANT button. When the GRANT button is clicked, CP pro-

© 2023 Information Processing Society of Japan

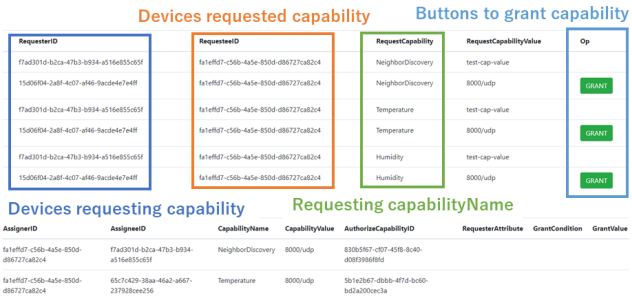


Fig. 10 WebUI of prototype implementation.

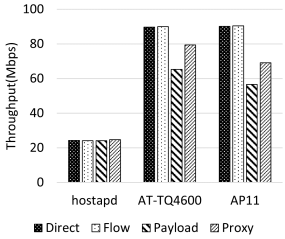


Fig. 11 Average throughput (iperf3).

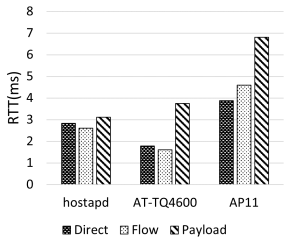


Fig. 12 Average RTT (ping).

cesses and applies a capability to PEP. Based on an applied capability, PEP configures an OpenFlow switch and twin devices. Access control is achieved with one click.

5.2 Performance Evaluation

In performance evaluation, we used hostpad, which is a software access point, Allied Telesis AT-TQ4600, which supports OpenFlow 1.3, and Aruba AP11, which is a low-cost AP for small businesses, with the firmware rewritten to OpenWrt and an Open vSwitch installed. For the PEP, ThinkPad X260 (Core i5-6200U (2Core/4Threads), 16 GB Memory) is used. Two devices (Raspberry Pi 4) are connected to the network, and each device has one authorized capability. Bandwidth was measured using iperf3, and latency was measured using ping. Measured values are average values in 120 seconds.

The results of the measurements are shown in **Fig. 11** and **Fig. 12**. Flow-based control has almost the same throughput as direct communication without control. Payload-based control is about 60 to 70% of that of direct communication. Proxy-based control is about 70 to 80% of that of direct communication.

The same tendency is observed for latency. The RTT of flow-based control is almost the same as direct communication. Payload-based control is degraded from 1.5 to 2 times that of direct communication. Proxy-based control is the same as payload-based control because ICMP packets in both methods go through the same path.

From these results, flow-based control is suitable for large

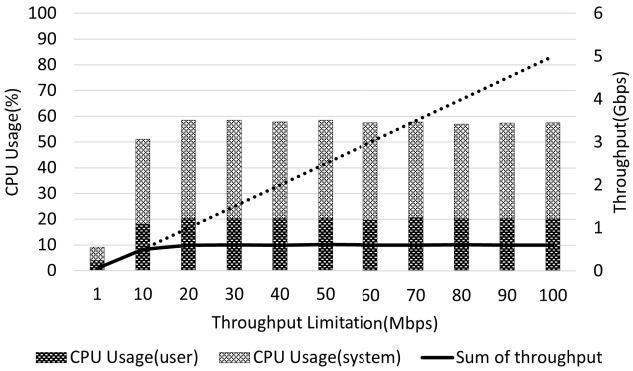


Fig. 13 CPU usage in payload-based control with iperf3 (A dotted line is ideal throughput).

data transfers such as video streams or latency-aware applications such as VoIP, and payload-based or proxy-based control is suitable for neighbor search and control messages like HTTP. Using the proper control method for each communication can minimize performance degradation. Also, the results show that our methods do not require expensive enterprise AP, and our methods also work well with low-const AP. This helps to lower the barrier to introducing the methods to real home networks.

5.3 Resource Consumption in PEP

We also measured the resource consumption in PEP. CREBAS provides payload-based and proxy-based controls with PEP. Payload-based and proxy-based controls require more computational resources. We measured CPU usage in PEP and confirmed that resource consumption is acceptable.

We conducted the measurements with the following scenario. PEP runs on a virtual machine (VM) with the same number of CPU cores and threads and memory capacity as the performance experiments. 2 VMs are connected to the PEP and they perform iperf3 and packet or message communications. From statistics [20], [21], [22], [23] about the number of devices in a home, we assume 8 devices (4 smart devices, 2 smartphones, 1 tablet, and 1 PC) as load. From this assumption, if each device communicates in a full mesh, the number of connections at the same time will be 28. Considering the increase in devices, we conducted the experiments with 50 parallel connections.

We conducted two experiments. One is iperf3 and the other is UDP packet or TCP message exchanging. In the first experiment, we varied the rate limitation of each iperf3 connection and measured CPU usage. In the second experiment, we varied the interval of exchange of UDP packets or TCP messages. From the investigation of IoT device communications [24], we used 1 KiB UDP packets and 10 KiB TCP messages. This size setting covers most communications for IoT devices.

Figure 13 and **Fig. 14** show the sum of the throughput of iperf3 and the CPU usage in payload-based and proxy-based controls. In a payload-based control (Fig. 13), it consumes about 50% to 60% of CPU and the throughput does not scale. This happens because the payload-based control needs to handle, parse, and check all packets in a single thread. In a proxy-based control (Fig. 14), it consumes CPU resources linearly with a throughput increase. This is because each TCP connection is handled parallelly in each

© 2023 Information Processing Society of Japan

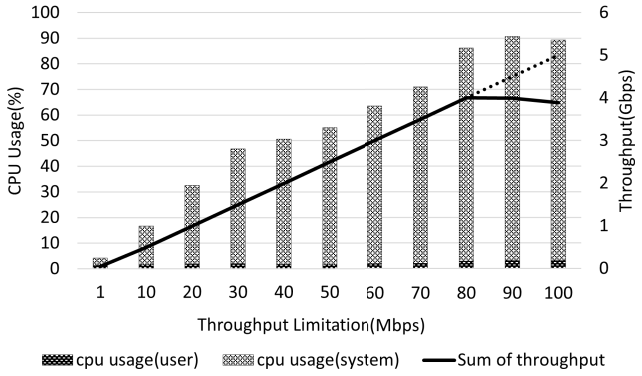


Fig. 14 CPU usage in proxy-based control with iperf3 (A dotted line is ideal throughput).

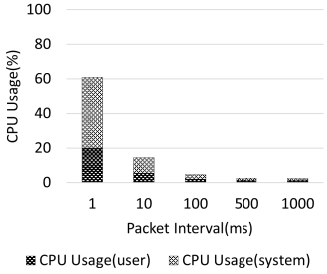


Fig. 15 CPU usage in payload-based control.

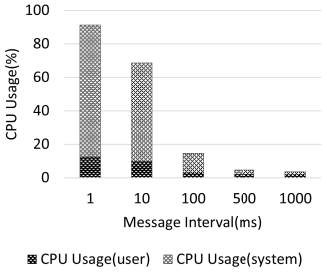


Fig. 16 CPU usage in proxy-based control.

thread in the proxy-based control. However, this control cannot adequately handle connections over 80 Mbps.

Figure 15 shows the CPU usage of the payload-based control with the exchange of UDP packets. CPU usage is less than 20% if the interval is longer than 10 ms. From the investigation [24], most devices send UDP packets less than 100 packets/second and 10 KiB/second. Therefore, payload-based control is expected to consume very few CPU resources.

Figure 16 shows the CPU usage of the proxy-based control to exchange TCP messages. CPU usage is greater than 60% when the interval is shorter than 10 ms. This is because TCP transparent proxy used in the proxy-based control needs to establish connections for every message exchange. However, most devices establish 1∼4 connections per second. Therefore, the proxy-based control is expected to also consume very few CPU resources if used to control TCP message exchanges.

From these results, payload-based and proxy-based control will consume a lot of CPU resources when they are used in inappropriate ways like large data transfers. Such communications should be controlled with flow-based control. If payload-based and proxy-based controls are used to control communications for controls that do not exchange large data, resource consumption is acceptable.

5.4 PoC with a Real Protocol

Attack Scenario

CREBAS aims to control communications among devices with protocols that do not have authentication and encryption mechanisms. Because of the absence of authentication, attackers can easily perform malicious operations like stealing information and making devices malfunction. CREBAS assumes not only completely malicious attackers but also suspicious devices that work correctly but try to steal information covertly. Such suspicious devices cannot be purged because users want to use the devices and users may not notice that devices are behaving incorrectly. CREBAS prevents attacks from such devices by controlling communications in units of operation.

As stated in Section 4, some protocols [8], [19] are performed over HTTP REST API. Previous works use flow-based control so that they can purge devices from the network, but they cannot block only malicious communications or operations. CREBAS prevents attacks from devices by controlling communications in units of operation using not only flow-based control but also payload-based and proxy-based control.

PoC with DIAL Protocol

We controlled Discovery and Launch (DIAL) protocol [8] using the CREBAS prototype system. The DIAL protocol is based on the Simple Service Discovery Protocol (SSDP) and DIAL REST Service. SSDP discovers devices in a network, and devices are controlled via HTTP REST API called the DIAL REST Service.

DIAL protocol does not have any access control mechanisms so that malicious devices can detect and control other devices in a home network. Access control methods with only flow-based control can mitigate some of these problems, but not enough. Flow-based control can only either allow or deny the entire device discovery. Therefore, it is impossible to allow for specific discovery packets. Also, since the REST Service uses HTTP, it is impossible to allow or deny only specific API operations with flow-based control.

As for SSDP, the response to a multicast discovery packet contains a Search Target (ST) header that identifies the service. Payload-based control can inspect the header in a payload. If the service in the ST header is permitted to discover, the packet and its response packets are passed through. As for DIAL REST Service, all operations are defined as HTTP REST API. Proxy-based control can inspect HTTP requests and control requests based on HTTP methods and URI. If the operation is permitted, the request is passed through, and if the operation is not permitted, the request is discarded.

We confirmed that the above control works with dial-reference [*1], a simulator of the DIAL protocol. As for the SSDP, only responses that contain the ST header with urn:dial-multiscreen-org:service:dial:1 are permitted, and the others are dropped. For the DIAL REST service, the capabilities are assigned to each of the Netflix-related operations listed in **Table 2**.

[*1] https://github.com/Netflix/dial-reference

© 2023 Information Processing Society of Japan

Table 2 An example of operations in DIAL.

Operation	HTTP Method and URI
Netflix Start	POST /apps/Netflix?clientDialVer=2.1
Netflix Stop	DELETE /apps/Netflix?clientDialVer=2.1
Netflix Hide	POST /apps/Netflix//hide?clientDialVer=2.1
Netflix Status	GET /apps/Netflix?clientDialVer=2.1

We confirmed that only authorized devices could discover other devices, and only permitted DIAL operation can be done.

6. Discussion

6.1 The Validity of Authorization Architecture

We discuss the proposed authorization architecture and access control method from the perspective of guidelines mentioned in Section 3.

(1) Allow fine-grained control

We propose the access control method with flow-based, payload-based, and proxy-based control. As mentioned in Section 5, these methods cover not only flow-oriented large data transfer but also messages transferred using TCP. Therefore, the limit of the granularity of the control relies on the granularity of the definition of capability and the implementation of twin devices. If capabilities are defined properly by device vendors, users only need to authorize some capabilities and fine-grained control is achieved.

(2) Plan for lending devices

A typical home has multiple users, and devices are lent or shared among them [13], [14], [15]. In our proposal, capabilities are issued to twin devices so that user-specific access control can be achieved by assigning twin devices for each user on a loaned physical device. In addition, there are cases where the scope of authorization of capabilities can change, such as capabilities that only the owner of the device can authorize, that family members can authorize, or that visitors can authorize. It is desirable to be able to configure a restriction of authorization for each user.

In a multiple-user system, the system needs to know who is using the device. Switching twin devices in CREBAS WebUI or the device's UI is one option, but it sometimes bothers users. Mechanisms are required, such as encouraging a user to switch a twin device when a lent device is connected to a user's wireless AP. Multiple-user system considering lending devices is a future issue.

(3) Include reactive policy creation

As shown in Fig. 9, users can manually authorize capability requests which were not automatically authorized with one click.

(4) Include logs

Since all capabilities are issued or delegated via CP, the capabilities used for access control are maintained on CP. Logs for authorization can be easily obtained from CP.

The logs of the actual network access control operations are based on a controller in the PEP and twin devices. In our implementation, a controller has logs for flow-based control. The logs contain what twin device was started, what capability was applied, and what flow was added to an OpenFlow switch. The twin devices have logs for payload-based and proxy-based control. The logs contain what packets or messages were received and how a twin device processed them. Since a controller and twin devices are just software, we can make a controller and twin devices to get more detailed logs and analyze logs.

(5) Reduce or eliminate up-front complexity

By using twin devices and automatic configuration of twin devices, access control can be achieved by simply connecting physical devices to an OpenFlow switch connected to the PEP. Therefore, there is no need to configure the settings or policies for each device. Users can control communications only by making some authorization decisions in Manual Authorization. Automatic Authorization is designed as a countermeasure to increase the number of authorization requests to users.

A profile system [16] can extend the Automatic Authorization. Capabilities that are often authorized and considered not to cause security vulnerabilities can be authorized automatically. With profiles to authorize such capabilities automatically, Automatic Authorization will be more usable.

(6) Support iterative policy specification

This can be easily achieved by updating the capabilities in physical devices or twin devices.

As described above, the architecture and access control method largely meet the requirements. However, some points need to be addressed in the future.

6.2 The Effectiveness of Automatic Authorization

Automatic Authorization is designed for cooperation among the same vendor's devices. In our authorization architecture, CREBAS needs to authorize and issue capabilities for all communications including device discovery. If a device wants to discover other devices, users are required to authorize each capability to the discoverer device. As an example, there are n devices, and each device wants to discover the other. Without Automatic Authorization, users need to authorize capabilities $n(n-1)$ times. If there are 10 devices on a home network and they want to discover each other, users must authorize them 90 times. This is the weakness of the capability model. To reduce the number of authorizations in the capability model, we designed Automatic Authorization. Automatic Authorization reduces the number of authorizations from $n(n-1)$ to *zero*. Users are not required to authorize many times and devices can discover each other.

6.3 Comparison with Previous Works

CREBAS provides fine-grained network access control to control communications in units of operations. However, there is a dilemma that the more fine-grained control is provided, the more effort is required for users to configure the control. If its operation is complex or burdensome, users will make misconfigurations or will not use such a system. Usability is very important and the access control system needs to care about that.

As stated in Section 2, previous works [1], [2], [3], [4], [5] do not discuss how to configure them or simply provide a profile-based system. They are not likely to be utilized in home networks and more work was needed to apply their work in real home networks. Some appliances (e.g., FortiGate, YAMAHA, SquidGuard, etc.) provide rule-based or profile-based access control. These require users to configure rules described by regular expressions or use 'black-box' profiles. Such setup processes are

© 2023 Information Processing Society of Japan

too burdensome for ordinary people.

CREBAS is designed to enable access control with one click based on the knowledge for usable access control system [11], 'One Click' means that CREBAS does not require users to configure complex policies and rules. CREBAS generally requires 2 steps to configure access control. Users just need to read the brief description of communication and click the GRANT button on WebUI to allow it, like permission management in smartphones.

This is achieved by authorization architecture based on capability model and network access control method including autoconfiguration mechanisms. The authorization architecture handles capabilities that have descriptions and specifications of communication and a policy for automatic authorization. Also, as described in Section 4, components including capabilities and twin devices are automatically configured using DHCP. Users do not need to care about network statuses and protocols like "What IP address is assigned to the device?". As shown in Section 5.4, we showed that the one click configuration with CREBAS works in a real environment, and users are not required to configure rules or policies.

However, users may make inappropriate authorization by mistake. To reduce such mistakes, we should design the user interface appropriately. The design of the user interface is beyond the scope of this paper. However, the capability-based authorization architecture will make it easy, because users do not need to define rules or policies. Also, the detection of mistakes by machine learning [25] will be easy. This is because all authorizations and controls are represented as capabilities like permissions in smartphones, not complex rules and policies. The system detects suspicious authorizations (capabilities) and recommends users revoke it will help to reduce misconfiguration.

6.4 The Limitations of CREBAS

CREBAS has two limitations coming from twin devices.

One limitation is that if a physical device does not have a twin device, CREBAS does not work. Users must develop a twin device themselves if a device does not have it. However, some devices may not have device vendor support or public protocol specifications, and this prevents users from developing twin devices. It is also true for previous works discussed in Section 6.3. They require the protocol to be public to define their rules or profiles. Device vendors should develop twin devices or make public their protocol to enable CREBAS and previous works to protect other devices.

The other limitation is the granularity of access control. The granularity of access control relies on twin devices. Twin devices may provide insufficient granularity control and capabilities. Such twin device is undesirable and device vendors should develop twin devices earnestly to protect their devices.

7. Conclusion

In this paper, we have proposed CREBAS which is a system that allows access control in a home network with one click. To realize this system, we have proposed capability-based authorization architecture and network access control methods. We have designed an authorization architecture according to the guidelines of access control in a home. Automatic Authorization using capability delegation automatically makes authorization decisions and reduces the burden on users. To provide sufficient granular access control with authorized capabilities, we have designed network access control with flow-based, payload-based, and proxy-based control. These methods enable more fine-grained access control than previous works. Twin devices process capability and control packets with minimal modification of existing devices. Finally, we have implemented a prototype system. The evaluation has shown that the performance overhead is acceptable, and the methods work with a real protocol.

References

[1] Feamster, N.: Outsourcing Home Network Security, *HomeNets '10*, pp.37–42, ACM (2010).

[2] Flores Moyano, R., Fernndez Cambronero, D. and Bellido Triana, L.: A User-Centric SDN Management Architecture for NFV-Based Residential Networks, *Comput. Stand. Interfaces*, Vol.54, No.P4, pp.279–292 (2017).

[3] Osman, A., Wasicek, A., Köpsell, S. and Strufe, T.: Transparent Microsegmentation in Smart Home IoT Networks, *HotEdge '20*, USENIX, pp.1–6 (2020).

[4] Mortier, R., Rodden, T., Lodge, T., McAuley, D., Rotsos, C., Moore, A., Koliousis, A. and Sventek, J.: Control and understanding: Owning your home network, *COMSNETS 2012*, pp.1–10 (2012).

[5] Lear, E., Droms, R. and Romascanu, D.: Manufacturer Usage Description Specification, RFC 8520 (2019).

[6] Vallati, C., Virdis, A., Mingozzi, E. and Stea, G.: Mobile-Edge Computing Come Home Connecting things in future smart homes using LTE device-to-device communications, *IEEE Consumer Electronics Magazine*, Vol.5, No.4, pp.77–83 (2016).

[7] ECHONET Consortium: The ECHONET Lite Specification Version 1.13 (2018) (online), available from ⟨https://echonet.jp/spec_v113_lite_en/⟩ (accessed 2023-02-14).

[8] Netfix, Inc.: DIAL Discovery And Launch protocol specification Version 2.2.1 (2021).

[9] McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S. and Turner, J.: OpenFlow: Enabling Innovation in Campus Networks, *SIGCOMM Comput. Commun. Rev.*, Vol.38, No.2, pp.69–74 (2008).

[10] Yao, Y., Basdeo, J.R., Kaushik, S. and Wang, Y.: Defending My Castle: A Co-Design Study of Privacy Mechanisms for Smart Homes, *CHI '19*, pp.1–12, ACM (2019).

[11] Mazurek, M.L., Arsenault, J.P., Bresee, J., Gupta, N., Ion, I., Johns, C., Lee, D., Liang, Y., Olsen, J., Salmon, B., Shay, R., Vaniea, K., Bauer, L., Cranor, L.F., Ganger, G.R. and Reiter, M.K.: Access Control for Home Data Sharing: Attitudes, Needs and Practices, *CHI '10*, pp.645–654, ACM (2010).

[12] Mazurek, M.L., Klemperer, P.F., Shay, R., Takabi, H., Bauer, L. and Cranor, L.F.: Exploring Reactive Access Control, *CHI '11*, pp.2085–2094, ACM (2011).

[13] Jang, W., Chhabra, A. and Prasad, A.: Enabling Multi-User Controls in Smart Home Devices, *IoTS&P '17*, pp.49–54, ACM (2017).

[14] Jacobs, M., Cramer, H. and Barkhuus, L.: Caring About Sharing: Couples' Practices in Single User Device Access, *GROUP '16*, pp.235–243, ACM (2016).

[15] Matthews, T., Liao, K., Turner, A., Berkovich, M., Reeder, R. and Consolvo, S.: "She'll Just Grab Any Device That's Closer": A Study of Everyday Device & Account Sharing in Households, *CHI '16*, pp.5921–5932, ACM (2016).

[16] Egelman, S., Brush, A.B. and Inkpen, K.M.: Family Accounts: A New Paradigm for User Accounts within the Home Environment, *CSCW '08*, pp.669–678, ACM (2008).

[17] Levy, H.M.: 1 - Capability- and Object-Based System Concepts, *Capability-Based Computer Systems*, Levy, H.M. (Ed.), Digital Press, pp.1–20 (1984).

[18] Gusmeroli, S., Piccione, S. and Rotondi, D.: IoT Access Control Issues: A Capability Based Approach, *IMIS 2012*, pp.787–792 (2012).

[19] Sony: IP Control BRAVIA Professional Displays (2019) (online), available from ⟨https://pro-bravia.sony.net/develop/integrate/ip-control/index.html⟩ (accessed 2023-02-14).

[20] F-Secure: THE CONNECTED HOME'S NEXT WAVE (2020) (online), available from ⟨https://blog-assets.f-secure.com/wp-content/uploads/2020/07/23093355/2020-07-02_FS_CHS-report_A4_RGB_final.pdf⟩ (accessed 2023-02-14).

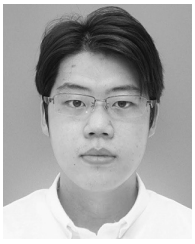
© 2023 Information Processing Society of Japan

[21] Communications and Information network Association of Japan: Usage Survey of Mobile Devices in 2020 (2020) (online), available from ⟨https://www.ciaj.or.jp/pressrelease2020/6280.html⟩ (accessed 2023-02-14).
[22] Ministry of Internal Affairs and Communications, Japan: WHITE PAPER 2021, Information and Communications in Japan (2021) (online), available from ⟨https://www.soumu.go.jp/johotsusintokei/whitepaper/eng/WP2021/2021-index.html⟩ (accessed 2023-02-14).
[23] Statistics Japan: Statistical Handbook of Japan 2021 (2021) (online), available from ⟨https://www.stat.go.jp/english/data/handbook/pdf/2021all.pdf⟩ (accessed 2023-02-14).
[24] Mainuddin, M., Duan, Z. and Dong, Y.: Network Traffic Characteristics of IoT Devices in Smart Homes, *ICCCN 2021*, pp.1–11 (2021).
[25] Peiravian, N. and Zhu, X.: Machine Learning for Android Malware Detection Using Permission and API Calls, *ICTAI 2013*, pp.300–305 (2013).

Editor's Recommendation

This paper proposes a capability-based network access control mechanism, which will suppress communication by malicious devices unintentionally connected to the home network. The paper was highly evaluated at the DICOMO 2021 symposium for demonstrating the effectiveness of the proposed method using a prototype implementation and thus is selected as a recommended paper.

(Chief examiner of SIGIOT Yoshiaki Kitaguchi)



Naoki Matsumoto is a student of the Graduate School of Informatics, Kyoto University. He received his B.E. degree from Kyoto University in 2021. His research interests include home network architecture and computation offloading.



Daisuke Kotani received his B.E., M.Inf. and D.Inf. degrees from Kyoto University in 2011, 2012 and 2016 respectively. Since 2016 he has been an assistant professor at Academic Center for Computing and Media Studies, Kyoto University. His research interests include Internet architecture, distributed systems and system security. He is a member of IEEE, ACM, IPSJ and IEICE.



Yasuo Okabe received M.E. and D.E. from Department of Information Science, Kyoto University in 1988 and in 1994, respectively. He has been a Professor of Academic Center for Computing and Media Studies, Kyoto University since 2002. His current research interest includes Internet architecture, network security an distributed algorithms. He is a fellow of IEICE and IPSJ, and a member of ISCIE, JSSST, IEEE, and ACM.

© 2023 Information Processing Society of Japan